

Supplementary IV

(Aggregate Signature Schemes and Cryptanalysis Implementation)

This project implements three certificateless aggregate signature schemes (LPCIA, Rajkumar, Xu) and six targeted cryptanalysis attacks. A total of 9 C code files are developed based on the PBC cryptography library. All schemes generate 40 user key pairs and 40 individual signatures.

For the Rajkumar and Xu schemes, attack algorithms rely on the Wagner algorithm (a probabilistic birthday search algorithm). To ensure deterministic attack results, we modify the hash generation logic and replace probabilistic Wagner searching with fixed hash summation constraints.

1 Source Code Files (Total: 9)

1.1 Original Aggregate Signature Schemes (3 Files)

- `LPCIA.c`: Implementation of the LPCIA aggregate signature scheme
- `Rajkumar.c`: Implementation of the Rajkumar aggregate signature scheme
- `Xu.c`: Implementation of the Xu aggregate signature scheme

1.2 Cryptanalysis Attack Programs (6 Files)

- `LPCIAA1.c`: Type A1 attack on the LPCIA scheme
- `LPCIAA2.c`: Type A2 attack on the LPCIA scheme
- `RajA1.c`: Type A1 attack on the Rajkumar scheme
- `RajA2.c`: Type A2 attack on the Rajkumar scheme
- `XuA1.c`: Type A1 attack on the Xu scheme
- `XuA2.c`: Type A2 attack on the Xu scheme

1.3 Wagner Algorithm Utility Codes

- `Test1.c`: Wagner algorithm for 128 hash values with $\text{sum} = 1$ (64-bit)
- `TestH.c`: Wagner algorithm for 128 hash values with a fixed random target sum

2 Function Descriptions

2.1 Original Schemes (`LPCIA.c`, `Rajkumar.c`, `Xu.c`)

Each program performs the following operations:

1. Generate 40 pairs of user keys
2. Generate 40 individual signatures
3. Verify 40 partial private keys
4. Verify 40 individual signatures separately
5. Aggregate 40 signatures and verify the aggregate signature

2.2 Attack on LPCIA (LPCIAA1.c, LPCIAA2.c)

- TA only generates the key pair for user (V_0).
- **A1 Attack:** Adversary forges public keys, identities, and pseudonyms for ($V_1 \sim V_{39}$)
- **A2 Attack:** Adversary cannot modify public keys, identities, or pseudonyms for ($V_1 \sim V_{39}$)

2.3 Attack on Rajkumar (RajA1.c, RajA2.c)

- TA only generates key pairs for users ($V_1 \sim V_{39}$)
- **A1 Attack:** Adversary forges public key, identity, and pseudonym for (V_0)
- **A2 Attack:** Adversary cannot modify public key, identity, or pseudonym for (V_0)

2.4 Attack on Xu (XuA1.c, XuA2.c)

- TA only generates key pairs for users ($V_1 \sim V_{39}$)
- **A1 Attack:** Adversary forges public key, identity, and pseudonym for (V_0)
- **A2 Attack:** Adversary cannot modify public key, identity, or pseudonym for (V_0)

3 Key Modification: Wagner Algorithm Replacement

The Rajkumar and Xu schemes are attacked using the Wagner algorithm, which is **probabilistic** and cannot guarantee success in every run.

To achieve deterministic and stable attack results, we modify the hash generation logic:

1. In RajA1.c and RajA2.c: Force the sum of hash values to be **1**
2. In XuA1.c and XuA2.c: Force the sum of hash values to be a **fixed independent hash value**
3. The original probabilistic Wagner algorithm implementations are preserved in Test1.c and TestH.c

4 Important Compatibility Notes

4.1 Differences in Hash Bit Length Design

- Tool codes: Test1.c / TestH.c adopt 64-bit hash length, which is designed for efficient verification of the Wagner algorithm.
- Attack codes: RajA1.c / RajA2.c adopt 256-bit hash length, aiming to maintain consistency with the standard parameters in the manuscript.

4.2 Incompatibility Conclusion

Due to the obvious difference in hash bit lengths, Test1.c cannot be directly compatible or mixed with RajA1.c and RajA2.c.

5 Screenshot Requirements

5.1 Original Schemes

Include three verification results:

1. Verification of 40 partial private keys
2. Verification of 40 individual signatures
3. Verification of the final aggregate signature

```

36th Key is generated!
37th Key is generated!
38th Key is generated!
39th Key is generated!
The 0 individual signature is valid!
The 1 individual signature is valid!
The 2 individual signature is valid!
The 3 individual signature is valid!
The 4 individual signature is valid!
The 5 individual signature is valid!
The 6 individual signature is valid!
The 7 individual signature is valid!
The 8 individual signature is valid!
The 9 individual signature is valid!
The 10 individual signature is valid!
The 11 individual signature is valid!
The 12 individual signature is valid!
The 13 individual signature is valid!
The 14 individual signature is valid!
The 15 individual signature is valid!
The 16 individual signature is valid!
The 17 individual signature is valid!
The 18 individual signature is valid!
The 19 individual signature is valid!
The 20 individual signature is valid!
The 21 individual signature is valid!
The 22 individual signature is valid!
The 23 individual signature is valid!
The 24 individual signature is valid!
The 25 individual signature is valid!
The 26 individual signature is valid!
The 27 individual signature is valid!
The 28 individual signature is valid!
The 29 individual signature is valid!
The 30 individual signature is valid!
The 31 individual signature is valid!
The 32 individual signature is valid!
The 33 individual signature is valid!
The 34 individual signature is valid!
The 35 individual signature is valid!
The 36 individual signature is valid!
The 37 individual signature is valid!
The 38 individual signature is valid!
The 39 individual signature is valid!
The e_left is [38782387533914297375094075052657079705357
818127, 137412958985023755697661452341668874904336623387
The e_right is [3878238753391429737509407505265707970535
2818127, 13741295898502375569766145234166887490433662338
]
The aggregate signature is valid!

```

5.2 Attack Programs (All 6 Files)

The TA only generates a partial private key for $\mathcal{A}_1$ in the LPCIA scheme, but $\mathcal{A}_1$ can still forge an aggregate signature for 40 vehicles.

```
spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pb
0th Key is generated!
The P4 is [29274349967653328199740190884031513226182532127
5, 1579547012917269532913178120891367633637490042475429261
The aggregate signature is valid!
```

The TA generates 40 partial private keys for $\mathcal{A}_2$ in the LPCIA, Rajkumar and Xu schemes, respectively, but $\mathcal{A}_2$ cannot modify their public keys. $\mathcal{A}_2$ still can forge an aggregate signature for 40 vehicles.

```
36th Key is generated!
37th Key is generated!
38th Key is generated!
39th Key is generated!
The P4 is [7421588103099975583150297924656390083904619
6, 245685683302868032251970178628507017859359808576356
The aggregate signature is valid!
```

The TA generates 39 partial private keys for $\mathcal{A}_1$ in Rajkumar's and Xu's schemes, respectively, but $\mathcal{A}_1$ still can forge an aggregate signature for 40 vehicles.

```
35th Key is generated!
36th Key is generated!
37th Key is generated!
38th Key is generated!
39th Key is generated!
The e_left is [188587802182270781301039928539498074
075024, 2382551023308027908098209071565419176507762
The e_right is [18858780218227078130103992853949807
7075024, 238255102330802790809820907156541917650776
]
The aggregate signature is valid!
```

6 Compilation Notes

All programs depend on the PBC library and GMP library.

6.1 Compilation Command

```
gcc -o LPCIA LPCIA.c -lpbc -lgmp -lcrypto
gcc -o LPCIAA1 LPCIAA1.c -lpbc -lgmp -lcrypto
gcc -o LPCIAA2 LPCIAA2.c -lpbc -lgmp -lcrypto
gcc -o Rajkumar Rajkumar.c -lpbc -lgmp -lcrypto
gcc -o RajA1 RajA1.c -lpbc -lgmp -lcrypto
gcc -o RajA2 RajA2.c -lpbc -lgmp -lcrypto
gcc -o Xu Xu.c -lpbc -lgmp -lcrypto
gcc -o XuA1 XuA1.c -lpbc -lgmp -lcrypto
gcc -o XuA2 XuA2.c -lpbc -lgmp -lcrypto
```

6.2 Execution Command

```
./LPCIA
./LPCIAA1
./LPCIAA2
./Rajkumar
./RajA1
./RajA2
./Xu
./XuA1
./XuA2
```